

WIPRO PYTHON AUTOMATION

REPORT-1

Module 1: Automation with Selenium

Name: Debanjali De

Enrollment Number: 12023002001076

Year: 4th Year

Branch: Computer Science and Engineering

Wipro Training Programme, 2026

TABLE OF CONTENTS

- 1. Introduction
- 2. Module 1 – Automation with Selenium
 - 2.1 Introduction and Different Flavors of Selenium
 - 2.2 Selenium WebDriver Introduction and Installation
 - 2.3 Locators and Object Identification
 - 2.4 Handling Different Controls on Web Page
 - 2.5 Handling Different Controls on Web Page – Advanced
 - 2.6 Handling Dynamic Elements
 - 2.7 Synchronization / Wait Type
 - 2.8 Take Screenshots and Exception Handling
 - 2.9 Advanced Interactions
 - 2.10 Module Summary
 - 2.11 Learning Outcomes
- 3. Conclusion

1. Introduction

Automation in software testing refers to the use of tools and software processes to perform repeatable testing activities with reduced manual effort. Web automation focuses on interacting with web applications, checking expected behaviour, and supporting consistent execution of repetitive test scenarios.

Python is commonly used as a supporting language for automation because of its readable syntax and broad ecosystem of testing and automation libraries. Selenium is specifically used for browser-based automation because it provides mechanisms for controlling supported browsers and interacting with web elements.

Module 1, “Automation with Selenium,” builds the theoretical foundation for Selenium-based web automation. It progresses from Selenium fundamentals and WebDriver architecture to locators, controls, advanced browser interactions, dynamic elements, synchronization, screenshots, exception handling, and data-oriented interactions.

2. Module 1 – Automation with Selenium

2.1 Introduction and Different Flavors of Selenium

What is Selenium?

Definition: Selenium is a suite of tools and technologies used to automate interactions with web applications through supported browsers.

Purpose: Selenium is used to automate browser actions such as navigating pages, locating web elements, entering information, interacting with controls, and validating browser behaviour.

Importance: Browser automation reduces repetitive manual work and supports repeatable functional and regression testing.

Why Selenium?

- It supports automation of web applications through real browsers.
- It supports commonly used browsers and cross-browser testing scenarios.
- It provides WebDriver-based browser control.
- It supports identification and interaction with common web elements.
- It can be integrated with programming languages, test frameworks, external data, and reporting workflows.

Selenium Components

Component	Definition	Purpose	Typical Use
Selenium IDE	A browser-based record-and-playback oriented Selenium tool.	Supports quick creation and execution of browser interaction flows.	Learning, exploratory automation, and simple recorded tests.
Selenium RC	An older Selenium remote-control approach used before WebDriver became the primary architecture.	Provided programmatic browser control through a server-based model.	Legacy Selenium automation; largely superseded by WebDriver.
Selenium WebDriver	A browser automation interface that controls browsers through browser-specific driver mechanisms.	Provides programmatic control of browser interactions.	Modern web UI automation and functional testing.
Selenium Grid	A Selenium component for running tests across multiple environments.	Supports distributed, parallel, and cross-browser execution.	Testing across different browser and machine configurations.

2.2 Selenium WebDriver Introduction and Installation

Introduction to Selenium 4 WebDriver

Definition: Selenium WebDriver is the browser-control component of Selenium that allows an automation program to communicate with and control a supported browser.

Purpose: WebDriver provides the main mechanism for performing browser interactions in Selenium-based automation.

How It Works: The automation program sends WebDriver requests, the appropriate browser-driver mechanism processes those requests, and the selected browser performs the requested operation.

Selenium 4 WebDriver

Selenium 4 is the modern generation of Selenium used with WebDriver. It provides the current WebDriver-based approach used for browser automation in this training.

WebDriver Architecture

WebDriver architecture describes the communication path between the automation program and the browser. The automation program communicates through the Selenium WebDriver interface, the appropriate browser-driver mechanism translates the requests for the selected browser, and the browser executes the resulting actions.

- Automation program – defines the browser operation required by the test.
- Selenium WebDriver interface – provides the standard browser-control layer.
- Browser driver – acts as the communication mechanism for the selected browser.
- Browser – performs the requested navigation, element interaction, or validation-related action.

Software Requirements

Selenium WebDriver automation requires a compatible Python 3 environment, Selenium software, a supported browser, and a compatible browser-driver mechanism. Compatibility among these components is important for stable browser execution.

Setting Up WebDriver

WebDriver setup involves preparing the required software components, making the appropriate browser-driver mechanism available, and ensuring that the automation environment can communicate with the selected browser.

Browser Drivers and Browser Support

A browser driver is the communication component that connects WebDriver requests with a specific browser. Selenium can be used with supported browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox, provided the corresponding browser environment is correctly configured.

Windows System Path

The Windows system PATH is an operating-system mechanism used to locate executable programs from different locations. For browser automation, appropriate driver configuration can make the required browser-driver executable discoverable to the automation environment.

Running Automation on Chrome, Edge, and Firefox

Cross-browser execution means running the same automation concept against different supported browsers. This helps identify browser-specific behaviour and supports broader compatibility testing.

2.3 Locators and Object Identification

Understanding Web Elements

Definition: A web element is an individual component of a web page represented in the page structure, such as a button, text box, hyperlink, checkbox, radio button, dropdown, image, or table.

Importance: Selenium must identify the intended web element before an interaction or validation can be performed. Element identification depends on information such as tags, attributes, attribute values, visible text, and relationships among elements.

Challenges in Web Element Identification

Web pages may contain repeated elements, dynamically generated attributes, changing page structures, nested elements, hidden elements, and elements that appear only after another action. These conditions can make a locator unreliable if it depends on unstable or non-unique information.

Locator Strategies

Locator	How It Identifies	Advantages	Limitations	Suitable Use
ID	Identifies an element through its id attribute.	Usually concise and specific when unique.	Can be unreliable when IDs are dynamically generated.	Elements with stable unique IDs.
Name	Identifies an element through its name attribute.	Simple and readable when unique.	May match multiple elements.	Form fields with stable name values.
Tag Name	Identifies elements by their HTML tag.	Useful for element categories.	Often matches many elements.	Finding groups of similar elements.
Link Text	Identifies a hyperlink using its visible text.	Readable and directly related to displayed content.	Depends on exact or appropriate link text.	Distinctive text links.
Class Name	Identifies an element through a class attribute.	Useful when a stable class uniquely identifies the target.	Classes are often shared by many elements.	Elements with distinctive classes.
CSS Selector	Uses CSS-style relationships and attribute patterns to identify elements.	Flexible and often concise.	Can become difficult to maintain if page structure changes.	Structured and pattern-based identification.

XPath	Uses a path or relationship-based expression to locate elements.	Very flexible for complex page structures.	Can become brittle when based on unstable page structure.	Complex relationships and dynamic identification.
-------	--	--	---	---

Find Element

Definition: Identifies a target web element that matches the selected locator strategy.

Why It Is Needed: It allows the automation process to work with a specific page element.

Important Consideration: The locator should identify the intended element reliably.

Find List of Elements

Definition: Identifies multiple page elements that match a locator condition.

Why It Is Needed: It is useful for collections such as menus, rows, links, or repeated controls.

Important Consideration: The returned collection may contain zero, one, or many matching elements.

CSS Selectors, Wildcards and Child Nodes

CSS selectors identify elements using CSS-style patterns involving tags, attributes, classes, identifiers, and structural relationships.

CSS Wildcard Selectors

Definition: A wildcard selector uses a pattern to match elements or attribute values when an exact value is not required.

Importance: Wildcards are useful when part of an attribute value is stable but another portion changes. They should still be specific enough to avoid unintended matches.

CSS Child Nodes

Definition: A child selector identifies an element according to its direct structural relationship with a parent element.

Importance: Child relationships are useful for structured menus, grouped controls, lists, and other hierarchical page components.

XPath

Definition: XPath is a path-based locator technique that identifies elements using their position, attributes, text, and relationships within the page structure.

Absolute XPath

An absolute XPath describes a complete path from the root of the document to the target element. It can be precise but is generally sensitive to changes in the page hierarchy.

Relative XPath

A relative XPath identifies an element from a more flexible point in the page structure using meaningful attributes, text, or relationships. It is generally more adaptable than a complete document path.

XPath Using Attributes

Attribute-based XPath identifies an element through one or more HTML attributes and their values. It is useful when an attribute provides a stable way to distinguish the target.

XPath using contains()

The contains concept is used when only part of an attribute value or text is stable. It is useful for elements whose values contain predictable but changing portions.

XPath using starts-with()

The starts-with concept identifies elements when the beginning of an attribute value or text is stable while the remaining portion may change.

XPath using Parent

Parent-based XPath uses the relationship between a target element and its parent to identify an element within a known structural context.

XPath using Sibling

Sibling-based XPath uses the relationship between elements that share a common parent. It is useful when one element can be identified reliably and the target is positioned relative to it.

2.4 Handling Different Controls on Web Page

Buttons

Definition: A button is an interactive control used to trigger an action.

Purpose / Conceptual Selenium Interaction: Automation identifies the button and interacts with it according to the page behaviour.

Common Challenge: Buttons may be disabled, covered by another element, or dynamically rendered.

Input Box

Definition: An input box accepts user-entered information such as text, search terms, usernames, or other values.

Purpose / Conceptual Selenium Interaction: Automation can identify the field and conceptually perform input and validation operations.

Common Challenge: Common issues include wrong locators, read-only fields, and fields not yet interactable.

Checkbox

Definition: A checkbox allows an option to be independently selected or cleared.

Purpose / Conceptual Selenium Interaction: Automation can identify its current state and interact with it when required.

Common Challenge: The automation should consider whether the checkbox is already selected.

Radio Button

Definition: A radio button allows selection of one option from a related group.

Purpose / Conceptual Selenium Interaction: Automation can select the intended option and validate its state.

Common Challenge: Only the appropriate option within the group should be selected.

Select Box

Definition: A select box presents a predefined set of choices.

Purpose / Conceptual Selenium Interaction: Automation identifies the control and selects the required option according to its available choices.

Common Challenge: Custom dropdowns may behave differently from standard HTML select controls.

Alert Box

Definition: An alert is a browser-level dialog used to display information or require a response.

Purpose / Conceptual Selenium Interaction: Automation must handle the browser dialog before continuing with page interaction.

Common Challenge: Unexpected dialogs can block further browser actions.

Date Picker

Definition: A date picker is a control that allows the user to choose a date.

Purpose / Conceptual Selenium Interaction: Automation identifies the date-selection interface and selects or validates the intended date.

Common Challenge: Calendar widgets may be dynamic, multi-page, or custom-built.

File Upload

Definition: A file-upload control allows a user to provide a file to a web application.

Purpose / Conceptual Selenium Interaction: Automation interacts with the upload control according to the application's supported behaviour.

Common Challenge: File paths, permissions, and custom upload interfaces can create challenges.

2.5 Handling Different Controls on Web Page – Advanced

Handling Alert Boxes

Browser dialogs may require acknowledgement or a response before page execution can continue. Automation must recognize the dialog state and handle it appropriately.

Handling Datepicker

Dynamic calendar widgets may require navigation across months or years and identification of a particular date. The automation concept involves understanding the widget structure and selecting the required date.

Handling File Upload

File-upload controls may use a standard file input or a custom interface. Automation must account for the application's upload mechanism and the expected file-selection behaviour.

Handling Multiple Windows / Tabs

Modern web applications may open additional browser windows or tabs. Automation must distinguish the browser contexts and move between them when required.

Working with Keyboard Events

Keyboard events represent actions such as typing, pressing navigation keys, selecting text, or using shortcut keys. They are useful when mouse-only interaction does not represent the intended user action.

Working with Scrollbar

Scrolling changes the visible portion of a page. It is important when an element is outside the current viewport or when content loads as the page is scrolled.

Working with Drag and Drop

Drag-and-drop interaction moves an element from one location to another. It is common in dashboards, upload areas, sorting interfaces, and visual editors.

Handling Iframes / Frames

An iframe is an embedded document inside a web page. Elements inside the embedded document belong to a different browsing context and require appropriate context handling.

Find the State of an Element

Element state describes conditions such as enabled or disabled. State checking helps determine whether an element is currently available for interaction.

Get the Text of an Element

Reading visible element text is useful for validation, content verification, and locating meaningful page information.

Get the Value of an Element Attribute

Reading an attribute value provides information stored in the HTML element, such as a value, label-related property, or configuration attribute.

2.6 Handling Dynamic Elements

Dynamic Elements

Definition: A dynamic element is a web element whose presence, attributes, position, content, or state may change during page execution.

Why It Matters: Dynamic behaviour can make fixed locators or immediate interactions unreliable. Automation therefore needs to identify stable characteristics and account for changes in page state.

WebTable

Definition: A WebTable is a structured group of rows and columns displayed within a web page.

Purpose: WebTables are commonly used for lists, reports, inventories, records, and other structured information.

Traversing Through WebTable

Table traversal means moving conceptually through rows and columns to locate a required record or cell. The process normally involves understanding the table structure, identifying the relevant row or column, and locating the required value.

Extraction and Validation

Extraction is the process of retrieving required information from a page structure. Validation compares the retrieved information with an expected value or condition to determine whether the observed behaviour is correct.

2.7 Synchronization / Wait Type

Why Synchronization?

Synchronization coordinates automation actions with the timing of the web application. Modern pages may load elements asynchronously, update content after user actions, or display controls only after a condition is satisfied.

Timing Problems

Timing problems occur when automation attempts an operation before the required element or page state is ready. They can cause intermittent failures even when the same test sometimes succeeds.

Implicit Wait

Definition: An implicit wait is a general waiting policy that instructs WebDriver to keep checking for elements for a specified period before reporting that an element could not be found.

Use: It provides a general synchronization mechanism for element lookup operations.

Explicit Wait

Definition: An explicit wait is a condition-based synchronization mechanism that waits for a specified condition to become true before continuing.

Use: It is useful when a particular page state, element state, visibility condition, or other condition must be satisfied.

Aspect	Implicit Wait	Explicit Wait
Scope	General element lookup behaviour	Specific condition or situation
Condition Control	Less specific	Condition-based and targeted
Typical Purpose	Provide a general waiting policy	Wait for a known page or element condition
Flexibility	Lower	Higher
Use in Dynamic Pages	Useful for general timing variation	Useful for targeted dynamic conditions

2.8 Take Screenshots and Exception Handling

Screenshots in Selenium Automation

Definition: A screenshot is a captured image of the browser's visible state at a particular point during automation.

Purpose: Screenshots provide visual evidence of page state, support debugging, help document important checkpoints, and can assist in understanding failures.

Generic Method for Taking Screenshots

A generic screenshot method is a reusable approach for capturing browser state from different points in an automation flow without designing a separate screenshot process for every test step.

- Use a consistent naming approach so captured images can be associated with the relevant test activity.
- Store screenshots in an organized location for review and reporting.
- Capture meaningful browser states, especially important checkpoints or failures.
- Keep screenshot handling reusable across different automation scenarios.

Exception Handling

Definition: Exception handling is the structured management of unexpected conditions that interrupt normal program or automation execution.

Importance in Selenium: Browser automation may encounter timeouts, missing elements, stale elements, intercepted clicks, non-interactable controls, unexpected dialogs, and browser-driver problems. Structured handling helps automation respond to such conditions in a controlled way.

Try–Catch–Finally Concept

Try: Represents the operation that may encounter an unexpected condition.

Catch: Represents the handling of an exception when an unexpected condition occurs. In Python, this role is provided by exception-handling constructs using `except`.

Finally: Represents cleanup or finalization activity that should be considered after the operation, whether execution succeeds or an exception occurs.

Common Selenium-Related Failure Conditions

- Element not found or not available at the expected time.
- Element is present but not interactable.
- Element becomes stale after the page is updated.
- Another element or overlay prevents the intended interaction.
- A browser dialog or context change blocks the expected operation.
- Browser-driver or browser compatibility problems affect execution.

2.9 Advanced Interactions

Different APIs to Read Data from Excel and JSON

Automation projects may use libraries or APIs to read structured test data from Excel workbooks and JSON documents. The purpose is to separate test data from browser interaction logic and support reusable data-driven testing.

Reading Data from Properties, CSV and XML Files

Properties-style files commonly store key-value configuration information. CSV files represent tabular row-and-column data, while XML represents structured hierarchical data. These formats can provide configuration or test data to an automation process.

Reading from Excel Sheet / JSON

Excel is useful for structured tabular test data, while JSON is useful for structured key-value and collection-based data. Both can supply values such as inputs, expected results, or configuration information.

Writing Data into Excel / JSON

Writing data allows automation processes to store generated information such as extracted values, results, status information, or other execution-related data for later review or reuse.

Mouse Hover Actions

Definition: A mouse hover action occurs when the pointer is positioned over a web element.

Purpose: Hover behaviour is commonly used to reveal menus, tooltips, secondary controls, or additional information. Automation must account for the element's visibility and the state that triggers the hover-dependent content.

Executing JavaScript Commands

Definition: JavaScript commands provide a browser-side mechanism for performing selected operations or interacting with page content through JavaScript.

Purpose: JavaScript can be useful for specific browser-side operations, page inspection, or controlled interactions when the training scenario requires it. It should be used purposefully when standard Selenium interactions are not sufficient.

2.10 Module Summary

Module 1 establishes the theoretical foundation for Selenium-based web automation. It covers Selenium and its major components, WebDriver architecture and installation concepts, browser support, locators and object identification, common and advanced web controls, dynamic elements and WebTables, synchronization and wait strategies, screenshots, exception handling, and advanced interactions with external data.

The sequence of the module moves from foundational concepts to increasingly complex browser-automation concepts while keeping the focus on reliable identification, interaction, synchronization, validation, and maintainability.

2.11 Learning Outcomes

- Understand the purpose and role of Selenium in web application automation.
- Differentiate Selenium IDE, Selenium RC, Selenium WebDriver, and Selenium Grid.
- Explain Selenium 4 WebDriver and the basic WebDriver architecture.
- Understand browser-driver communication and the requirements for browser automation.
- Identify web elements using ID, Name, Tag Name, Link Text, Class Name, CSS selectors, and XPath.
- Differentiate absolute and relative XPath and understand attribute- and relationship-based XPath concepts.
- Understand the behaviour of common and advanced web controls.
- Explain dynamic elements, WebTables, traversal, extraction, and validation.
- Understand the purpose of synchronization and the difference between implicit and explicit waits.
- Explain screenshots as evidence and debugging support.
- Understand exception handling and the Try–Catch–Finally concept in automation.
- Understand external data handling through properties, CSV, XML, Excel, and JSON.
- Understand mouse hover actions and the role of JavaScript commands in browser automation.

3. Conclusion

Module 1, “Automation with Selenium,” provides a structured theoretical foundation for web application automation. It begins with the role of Selenium and WebDriver and progresses through element identification, browser controls, dynamic content, synchronization, screenshots, exception handling, and advanced data-oriented interactions.

This report is intentionally a theory-only document. It is based on the Module 1 syllabus and is independent of any Python source-code folder, practical implementation, assignment file, execution result, GitHub structure, or project-specific screenshots.